MegaMOO Developer Manual

MegaMOO is a **scratch-built native-Python codebase** in the tradition of LambdaMOO — a persistent, multiplayer, fully programmable text world, **not a line by line port of the C LambdaMOO source**. The MegaMOO server is an original implementation that adopts the MOO paradigm — numbered objects, single-parent prototype inheritance, natural language parsing and on-the-fly verb programming live inside the game — realized idiomatically in roughly **1.4 MB** of modern Python — server, entry point, and verb library — built on `asyncio` with **zero third-party dependencies**.

This manual is written for developers — people who will **build worlds**, **program verbs**, or **operate the server** — rather than for players. It explains not just *how* to use each subsystem but *why* it is built the way it is, with references into the source so you can follow the engine end to end.

> The single defining idea: in a classic MOO you program in a purpose-built MOO # > language. In MegaMOO, **the in-world programming language is Python itself.** Every verb — from `look` to `@dig` — is a plain `.py` file with the full standard library available, executed live against a persistent object database.

How to read this manual

Document	Audience	What it covers
01 — Architecture	Everyone	The object model, the request lifecycle from socket to verb to disk, persistence, and the design philosophy. Start here.
02 — Writing Verbs	Verb authors / programmers	The injected namespace, the source preprocessor, `$` constants, matching, custom parsers (verb types), emit substitution, the task system. The heart of the engine.
03 — Building Worlds	Builders / staff	The 64-command `@`-toolkit: rooms, exits, objects, descriptions, properties, and in-game verb programming, with a worked example.
04 — Operations	Operators	Running the server, command-line flags, configuration, the JSON API and MCP integration, persistence/backups, and the permission ladder.
05 — Engine Systems	Verb authors / programmers	The runtime systems verbs plug into: hooks (lifecycle events), the ticker (scheduled verbs), and the effects system.
06 — The Prototype Library	Builders / programmers	The shipped base objects — rooms (+ the compass), exits, containers and furniture — and their properties and verbs.
07 — The Help System	Builders / staff	The in-game `help` command: its resolution ladder, the `#34` topic store (articles and categories), self-documenting verb docstrings, and object introspection (`help #obj`).

The engine at a glance

Subsystem	Module	Responsibility
Server core	`moo/server.py`	Event loop, single-threaded verb execution via a one-worker executor, context propagation, graceful shutdown/restart
Networking	`moo/network.py`	Telnet/WebSocket connections, protocol negotiation, color and wrapping
Database	`moo/database.py`	SQLite persistence, lazy object cache, checkpointing
Object model	`moo/objects.py`	Inheritance, properties as native Python attributes, flags, tags
Parser	`moo/parser.py`	Command → verb / direct-object / preposition / indirect-object resolution
Verbs	`moo/verbs.py`	Verb compilation, preprocessing, caching, dispatch
Namespace	`moo/verb_namespace.py`	Builds the local scope every verb runs in
Matching	`moo/match_utils.py`	`pmatch` / `bmatch` natural-language object matching
Tasks	`moo/tasks.py`	Time-limited execution, `suspend` / `fork`, scheduling
Effects	`moo/effects.py`	Tick-driven, persisted timed-effect scheduling
Permissions	`moo/permissions.py`	Wizard/programmer/owner hierarchy, quotas
Builtins	`moo/builtins.py`	The function library injected into every verb's namespace
Strings	`moo/string_utils.py`	Gender-aware emit/pronoun substitution

World content lives in `moo verbs/<object-number>/<verb-name>.py` — 300+ verb files covering player commands and the staff/builder toolkit.

> **Scope.** This manual documents MegaMOO the engine and its generic tooling. The repository also contains a specific game world built on the engine, with its own systems (character progression, combat, equipment, and so on). That game's design and content are

> documented separately — this manual stays engine-only so the engine's capabilities are clear on their own.

What a verb looks like

`moo verbs/17/jump.py` — the `jump` command available in every in-character room:

```
"""
Jump across a gap or obstacle.

Usage: jump <exit>
"""

if not args:
    pobj.msg("Jump what?")
    return

# RT check
if (getattr(pobj, 'rt', None) or 0) > 0:
    pobj.msg("You must wait.")
    return

pos = getattr(pobj, 'position', 0) or 0
if pos:
    pobj.msg("You can't do that in your current position.")
    return

# Match exit in room contents
exit = pmatch(dobj, pobj, list(pobj.location.contents))
if not exit or not getattr(exit, 'is_exit', False):
    pobj.msg("Jump what?")
    return

if getattr(exit, 'jumpable', False):
    call_verb(exit, 'invoke')
elif getattr(exit, 'climbable', False):
    pobj.msg("You have to climb that!")
else:
    pobj.msg("You can't jump that!")
```

No imports, no boilerplate. The verb namespace arrives pre-loaded with the acting player (`pobj`), the parsed command text (`args` , `dobj` , `iobj` — all strings), object matching (`pmatch` , which turns that text into objects), cross-object calls (`call_verb`), and the rest of the builtin library. Objects are matched the way a player thinks — `jump gap` , `jump 2nd rope` — and the verb reads top to bottom like the action it performs. Writing Verbs explains every name in that namespace.

Design philosophy

MegaMOO is built with a deliberate set of trade-offs: it stays faithful to the MOO paradigm, keeps the engine small and dependency-free, and optimizes for the way text worlds are actually built and run. Each design choice below is stated with what it buys and what it costs.

1. **Faithful to the MOO paradigm.** The model is the product, not something approximated on top of a general-purpose framework. MegaMOO implements **prototypal, single-parent inheritance** over numbered objects directly; **in-game commands live on rooms** (the OOC/IC room parents) and dispatch to a `<verb>_` hook on matched objects inside verbs for per-object behavior; a real **natural-language parser** resolves articles, ordinals (`get 2 sword`), adjectives, possessives, and prepositions the way a player actually types. Nothing is bent to fit a class-per-table ORM. *Cost:* you don't inherit a framework's web/admin scaffolding — you build exactly what you want instead.
2. **Python is the language — hot-coded live, in-game.** Rather than implement a MOO-code interpreter, MegaMOO runs verbs as real Python with a thin preprocessor and a curated builtin library — the full standard library, native lists/dicts in properties, no DSL to learn. Verbs are plain files you can **write and edit from inside the running game** with the `@program` verb: on save it compiles and installs the verb into the live world (and writes the file to disk) in one step — **no reload, no restart, no build.** The change is live on the next invocation. The reverse direction is hot too: a background watcher picks up verb files edited on disk within a couple of seconds. Because the language is Python, the barrier to contributing is low and the pool of potential authors is everyone who already knows Python.
3. **Run many worlds at once.** A database is a **startup argument** (`megamoo.py <db> [port]`), so a development, staging, or experimental world runs **side by side with the live game** — each its own process and port — with no per-world install ceremony. Spinning up a scratch copy to test verbs and then promoting the work is trivial.
4. **Right-sized, not over-engineered.** Single-process, single-writer verb execution plus an embedded database is the correct fit for a text world's real load — commands are tiny and serialized, which sidesteps an entire class of data races for free. *Honest trade-off:* MegaMOO is not made to build a 10,000-player, multi-server MMO. MegaMOO is a modern Python and AI-enabled codebase engineered for building text-based multiplayer virtual worlds. Classic MOOs ran worlds of tens of thousands of rooms with hundreds of concurrent players on 1990s hardware; modern single-process hardware has magnitudes more headroom than any text world needs.
5. **Persistence without ceremony.** Objects live in SQLite (WAL mode for crash recovery) behind a lazy LRU cache, written through automatically when properties change. The hot working set stays in memory like a classic in-RAM MOO, but with durable, crash-safe persistence and **no**

code reload or stop-the-world checkpoint freeze. The world survives restarts without an explicit save step.

6. **Zero required dependencies.** The core engine is ~1 MB of pure Python on the standard library alone — no Django, no Twisted, no install tree to secure or upgrade. It is trivial to deploy, easy to read end to end, and (now that `asyncio` is in the stdlib) gives up little that a heavyweight framework would provide for a server of this kind.
7. **Accessibility is a feature of the engine, not a bolt-on.** MegaMOO is developed by a C1–C2 quadriplegic programmer using a head-pointer and typing on an on-screen keyboard at about 30 words per minute. A per-player `screenreader` mode strips all ANSI color and decoration; ANSI-aware wrapping measures visible width so escape sequences never break layout; server-side wrapping can be disabled entirely (`WRAP_WIDTH = 0`) for clients that reflow text themselves. Text worlds were the original accessible online games, and MegaMOO treats keeping them that way as part of its job.

Versions and conventions used in this manual

- **Engine version:** `0.10.0-beta20`, the single source of truth being `SERVER_VERSION` in `moo/globals.py` (printed by the startup banner and `megamoo.py --version`). The engine has not yet been load- or play-tested, so it is pre-release by design.
- **Object references** are written `#N` (e.g. `#3` is the staff base character). The core object hierarchy is summarized in Architecture.
- **Source references** are written `module.py:line` or `moo verbs/<obj>/<verb>.py` and point at the current tree.
- **Permission levels** are written `gm1` … `gm5`; see the permission ladder.

License

MIT — see LICENSE.